31. Design Teams Video Conferencing

Key Terms:

Term	Full Name	What it does
WebRTC	Web Real-Time Communication	Browser API for peer-to-peer audio/video
SFU	Selective Forwarding Unit	Server that forwards video without transcoding
MCU	Multipoint Control Unit	Server that mixes/transcodes all streams
TURN	Traversal Using Relays around NAT	Relay server when direct connection fails
ICE	Interactive Connectivity Establishment	Protocol to find best connection path
Simulcast	Multiple quality streams	Sender encodes multiple resolutions
NAT	Network Address Translation	Router hiding local IPs behind public IP

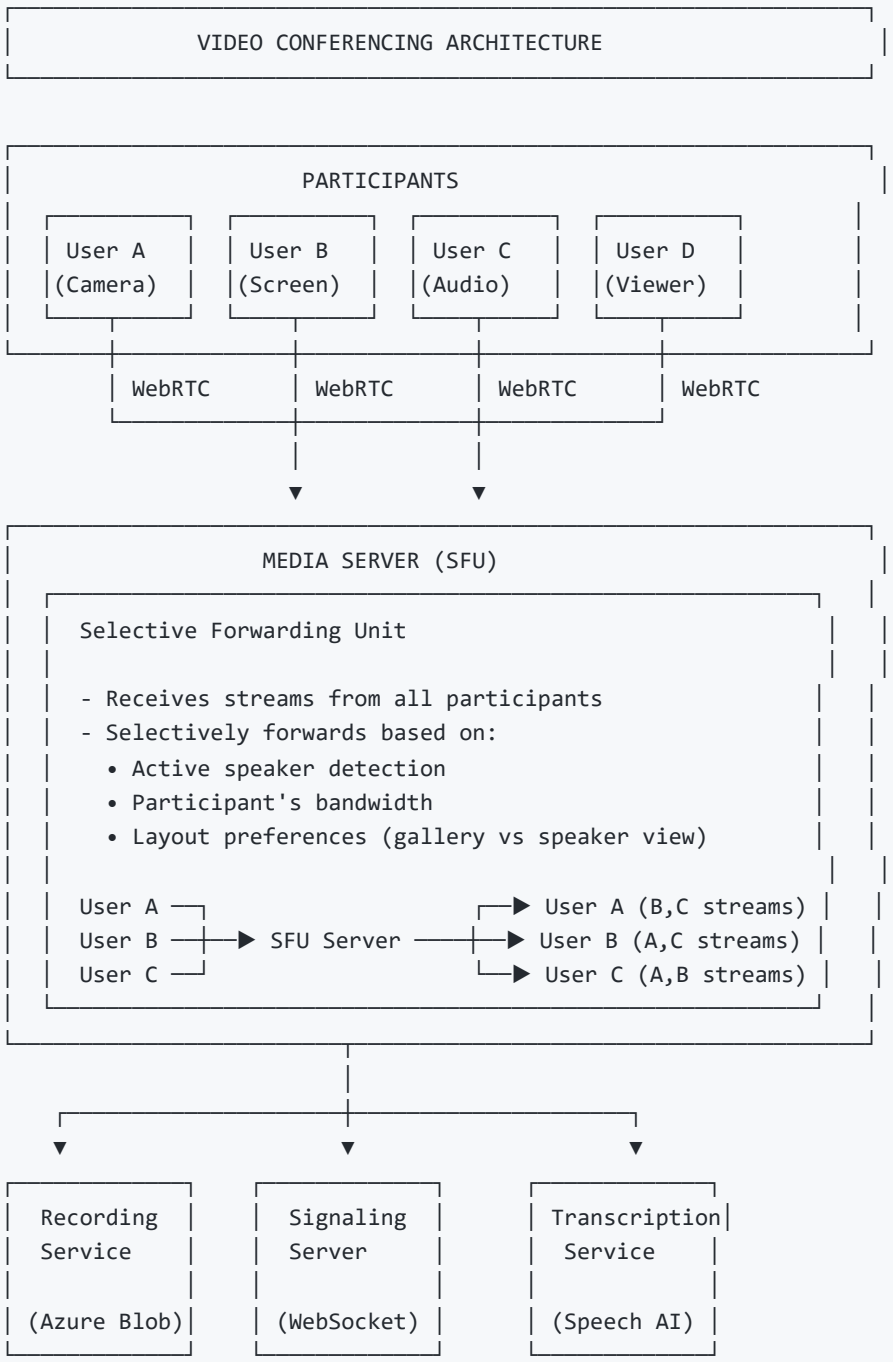
Problem Statement

Design a video conferencing system like Microsoft Teams Meetings that supports thousands of concurrent meetings with features like screen sharing, recording, and live captions.

Clarifying Questions to Ask

1. **Scale**: Max participants per meeting? Concurrent meetings globally?
2. **Features**: Video, audio, screen share, recording, live captions?
3. **Latency**: Max acceptable delay? (<150ms for real-time?)
4. **Quality**: Adaptive bitrate? Resolution options?
5. **Recording**: Where stored? Who can access?
6. **Breakout rooms**: Support splitting into smaller groups?
7. **Integration**: Calendar, chat, file sharing during meeting?

High-Level Design



API Design

Meeting APIs

Method	Endpoint	Description
`POST`	`/api/v1/meetings`	Create a meeting
`POST`	`/api/v1/meetings/{meetingId}/join`	Join a meeting

`POST`	`/api/v1/meetings/{meetingId}/leave`	Leave a meeting
`GET`	`/api/v1/meetings/{meetingId}`	Get meeting details
`GET`	`/api/v1/meetings/{meetingId}/participants`	List participants

Media Control APIs

Method	Endpoint	Description
`PUT`	`/api/v1/meetings/{meetingId}/media`	Toggle audio/video
`POST`	`/api/v1/meetings/{meetingId}/screenshare/start`	Start screen sharing
`POST`	`/api/v1/meetings/{meetingId}/screenshare/stop`	Stop screen sharing
`PUT`	`/api/v1/meetings/{meetingId}/layout`	Change video layout

Signaling WebSocket Events

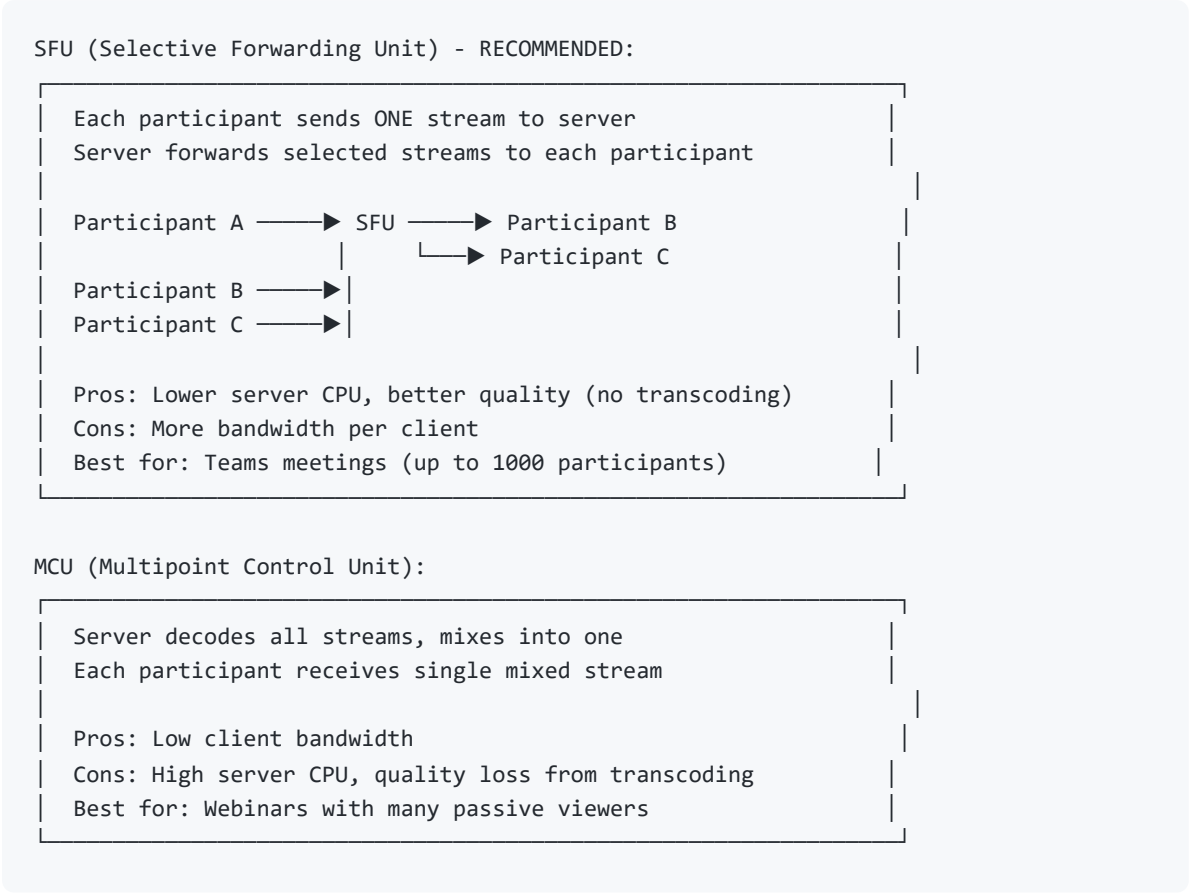
Event	Direction	Payload
`participant.joined`	Server → Client	`{ userId, name, mediaState }`
`participant.left`	Server → Client	`{ userId }`
`media.offer`	Client → Server	`{ sdp }`
`media.answer`	Server → Client	`{ sdp }`
`media.ice`	Both	`{ candidate }`
`media.state`	Server → Client	`{ userId, audio, video }`

Key Components

Component	Purpose	Technology
Signaling Server	Session setup, ICE negotiation	WebSocket
SFU (Selective Forwarding)	Route media streams	Mediasoup, Janus
TURN Server	NAT traversal relay	CoTURN
Recording Service	Capture and store meetings	FFmpeg + Blob Storage
Transcription	Live captions	Azure Speech Services

Quality Monitor	Adapt to network conditions	Custom metrics

Media Architecture: SFU vs MCU



Scalability Considerations

- **Regional SFU servers**: Deploy media servers close to participants
- **Cascading**: Connect SFUs across regions for global meetings
- **Simulcast**: Sender transmits multiple quality levels
- **Bandwidth estimation**: Adapt quality based on network conditions
- **Large meetings**: Switch to broadcast mode for 100+ participants

Trade-offs

Decision	Option A	Option B	Recommendation
Architecture	SFU	MCU	**SFU** - better quality
Protocol	WebRTC	Custom	**WebRTC** - browser native
Quality	Fixed	Adaptive	**Adaptive** - handles poor networks

Recording	Client-side	Server-side	**Server** - reliable

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
WebRTC	Peer-to-peer media streaming protocol	Tech Guide
SignalR	Real-time signaling and session management	Tech Guide
CDN	Media relay and content distribution	Tech Guide

How to Explain It Well

> **Opening**: "Video conferencing needs low latency and adaptive quality. I'll use SFU architecture with WebRTC for peer-to-peer media."

> **Key talking points**:
>
> - "SFU: each user sends one stream, server selectively forwards"
> - "Simulcast: sender transmits 3 qualities, receiver picks based on bandwidth"
> - "TURN servers for NAT traversal when direct connection fails"
> - "Active speaker detection reduces streams to forward"
> - "Recording is server-side composite for reliability"
